

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 4

Student Name: Nimidithalli Nandu

Roll No.: A24126552100

Date: 31-08-2026

1. TITLE

Design of an Interactive To-Do List Application using JavaScript DOM Manipulation and Event Listeners

2. AIM

To build an interactive To-Do List web application utilizing JavaScript event listeners (`addEventListener`), dynamic DOM element creation (`createElement`, `appendChild`), element removal (`remove`), and class toggling (`classList.toggle`).

3. OBJECTIVE

The objectives of this experiment are:

- To implement event-driven programming using JavaScript event listeners (`addEventListener`).
- To create and append new DOM elements dynamically (`createElement`, `appendChild`).
- To manipulate CSS class lists (`classList.toggle`) for task completion states.
- To manage DOM element removal (`remove()`) and handle empty state messages.

4. THEORY

JavaScript DOM manipulation allows dynamic modification of web documents without requiring page reloads. Methods like `document.createElement()` construct new HTML nodes in memory, while `appendChild()` attaches them to the DOM tree. Event handling via `addEventListener()` attaches execution callbacks to DOM elements. Element styling states can be toggled using `classList.toggle()`, and elements can be detached using `remove()`.

5. ALGORITHM / PROCEDURE

1. Create `index.html` and define document structure with CSS styles for task cards and completion states.
2. Design input field (`taskInput`), add button (`addButton`), and container (`taskList`) with empty message.
3. Add a click event listener to `addButton` using `addEventListener("click", callback)`.
4. Read input text from `taskInput.value` and validate non-empty string.
5. Hide the empty task list message (`emptyMessage.style.display = "none"`).
6. Dynamically create task container (`<div>`), text span (`<span>`), complete button, and delete button.
7. Attach click event listener to Complete button to toggle `.completed` class on text span (`classList.toggle`).
8. Attach click event listener to Delete button to remove task node (`task.remove()`) and restore empty message if list is empty.
9. Append text and action buttons to task container, append task container to task list, and clear input field.
10. Test and verify adding, completing, and deleting tasks in a web browser.

6. CODE EXPLANATION

The HTML file contains styled input fields, buttons, and task containers. In JavaScript, DOM references are assigned to variables (taskInput, addButton, taskList, emptyMessage). Clicking addButton checks for input, hides emptyMessage, creates a task card <div> containing a and two <button> elements ("Complete" and "Delete"). The Complete button toggles a line-through CSS class (.completed), while the Delete button removes the task node (task.remove()) and checks if emptyMessage should be displayed.

7. PROGRAM

index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>My To-Do List</title>
  <style>
    body {
      font-family: Arial;
      background-color: #9ea93a;
      text-align: center;
      margin: 40px;
    }
    h1 {
      color: #174a7e;
    }
    input {
      width: 350px;
      padding: 10px;
      border: 1px solid gray;
      border-radius: 5px;
    }
    button {
      padding: 10px 15px;
      margin: 5px;
      border: none;
      border-radius: 5px;
      background-color: #174a7e;
      color: white;
    }
    #taskList {
      width: 500px;
      margin: 25px auto;
    }
    .task {
      background-color: white;
      border: 1px solid #aaa;
      padding: 10px;
      margin: 10px;
      border-radius: 5px;
    }
    .completed {
      text-decoration: line-through;
      color: gray;
    }
    .delete {
      background-color: #1a1447;
    }
    #emptyMessage {
```

```

        color: gray;
    }
</style>
</head>
<body>
    <h1>My To-Do List</h1>
    <input type="text" id="taskInput" placeholder="Enter a task">
    <button id="addButton">Add Task</button>
    <div id="taskList">
        <p id="emptyMessage">No tasks available.</p>
    </div>
    <script>
        var taskInput = document.getElementById("taskInput");
        var addButton = document.getElementById("addButton");
        var taskList = document.getElementById("taskList");
        var emptyMessage = document.getElementById("emptyMessage");
        addButton.addEventListener("click", function() {
            var taskText = taskInput.value;
            if (taskText == "") {
                return;
            }
            emptyMessage.style.display = "none";
            var task = document.createElement("div");
            task.className = "task";
            var text = document.createElement("span");
            text.innerHTML = taskText;
            var completeButton = document.createElement("button");
            completeButton.innerHTML = "Complete";
            var deleteButton = document.createElement("button");
            deleteButton.innerHTML = "Delete";
            deleteButton.className = "delete";
            completeButton.addEventListener("click", function() {
                text.classList.toggle("completed");
            });
            deleteButton.addEventListener("click", function() {
                task.remove();
                if (taskList.children.length == 1) {
                    emptyMessage.style.display = "block";
                }
            });
            task.appendChild(text);
            task.appendChild(completeButton);
            task.appendChild(deleteButton);
            taskList.appendChild(task);
            taskInput.value = "";
        });
    </script>
</body>
</html>

```